Introduction to Prolog

Darko Bozhinoski,
Ph.D. in Computer Science

What is Prolog?

Theoretical Foundations

Prolog was developed based on formal logic and mathematical theories of computation.

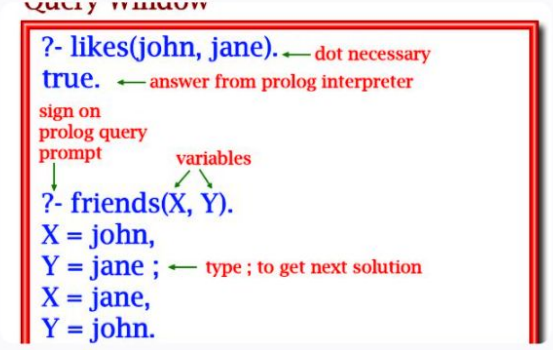
Declarative Language

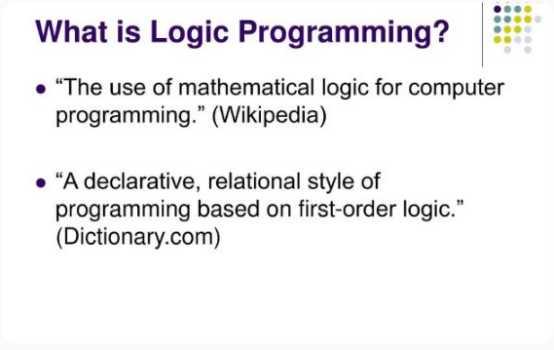
Prolog is a declarative language, which means you describe the problem and let the computer figure out how to solve it.

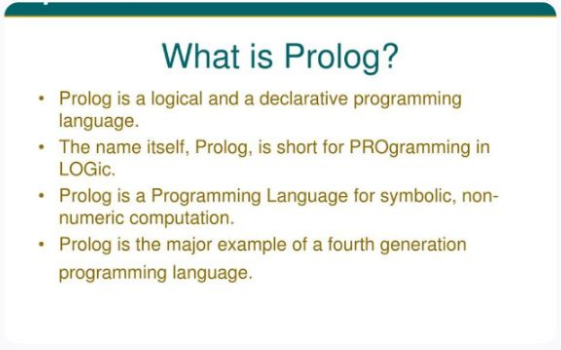
Pattern Matching

Prolog uses pattern matching to compare the input with the knowledge base and find solutions to problems.

Basic Concepts of Prolog







Syntax and Structure

Prolog programs are composed of facts and rules that follow a specific syntax and structure.

Logic Programming

Logic programming is based on rules, which consist of a head and a body that define relationships between objects.

Interpreters

Prolog programs are run by an interpreter that matches queries to the knowledge base and produces results.

Three Constructs in Prolog

- Facts
- Rules
- Queries

Working with Facts and Rules in Prolog

Facts

Facts are statements that are always true in the knowledge base, which can be used to build further relationships between objects.

Rules

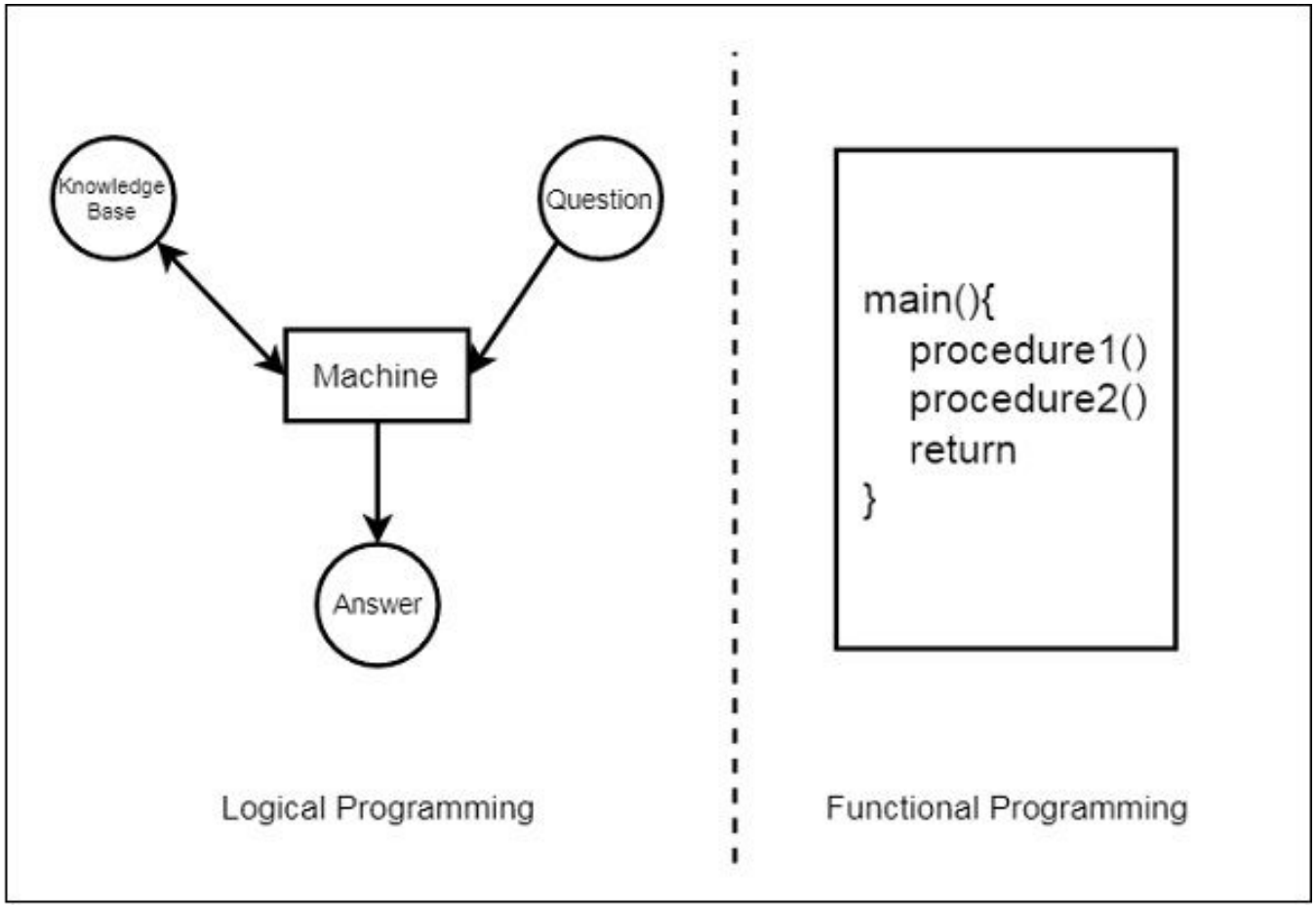
Rules define relationships between objects and can be used to solve problems by querying them with specific input.

KNOWLEDGE BASE

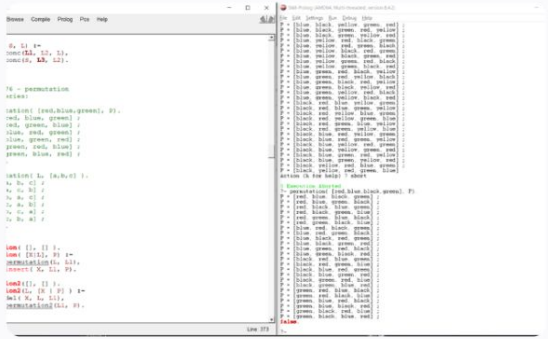
Facts and Rules together comprises a knowledge base of the program.

Queries are used on the knowledge base.

Syntax: Queries end with a “full stop”.

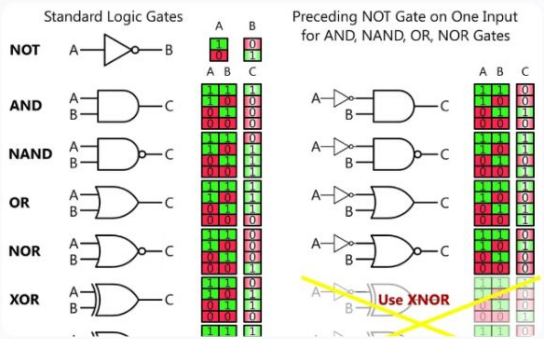

Knowledge Base
Question
Machine
Answer
Logical Programming
main(){
procedure1()
procedure2()
return
}
Functional Programming

Using Prolog for Logic Programming



Rule-Based Systems

Prolog is great for creating rule-based systems that can be used for problem-solving in various fields.



Logic Circuits

Prolog can simulate the behavior of digital logic gates and circuits, which can be used for testing logic designs.



Expert Systems

Prolog is widely used for creating expert systems that can provide automated advice and decision-making.

Disadvantages of Prolog

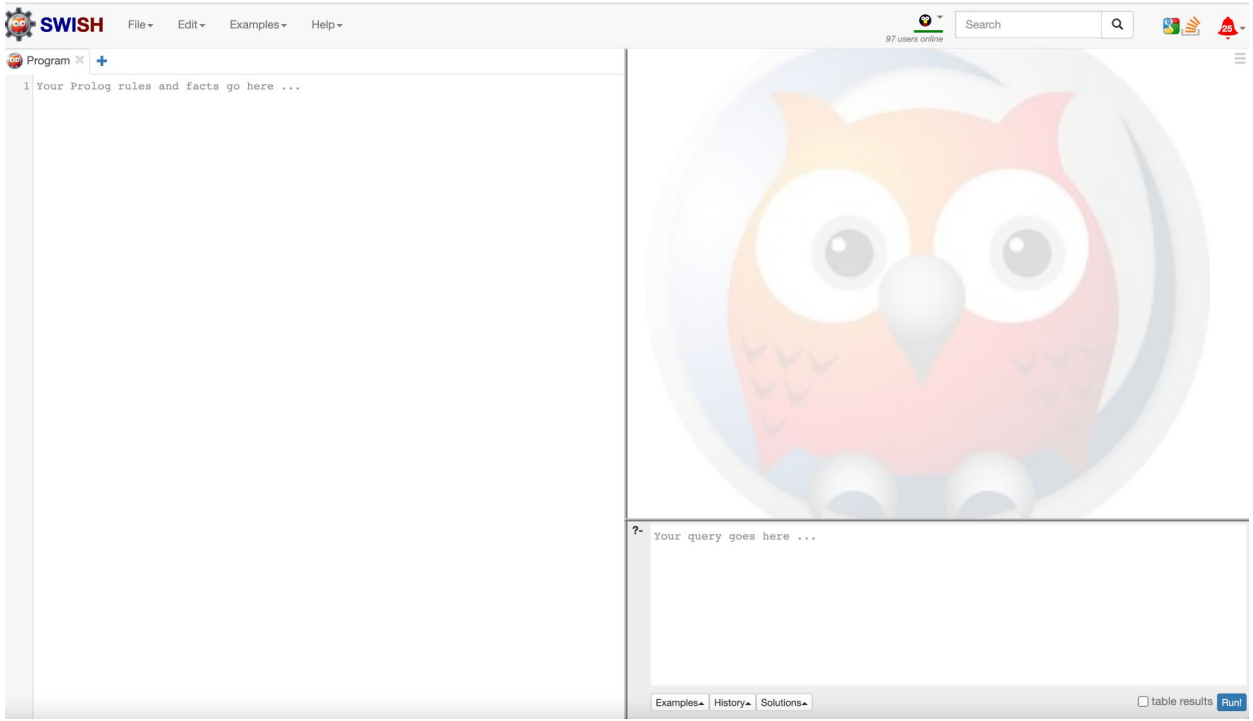
- Prolog does not have good Input/Output features compared to other languages like LISP.
- Prolog cannot handle statements where one of several things is true, but you don't know which.
- Prolog can be less performant than other languages, particularly for tasks that require a lot of computation.

SWI-Prolog

- SWI-Prolog runs on Windows, Macintosh and L inux platforms.
- Can be used online: https://swish.swi-prolog.org/

DEMO

- https://swish.swi-prolog.org/



Example

- Lili is happy if she dances.
- Tom is hungry if he is searching for food.
- Andrew and Ilnar are friends if both of them love to play football.
- Ryan will go to play if school is closed, and he is free.

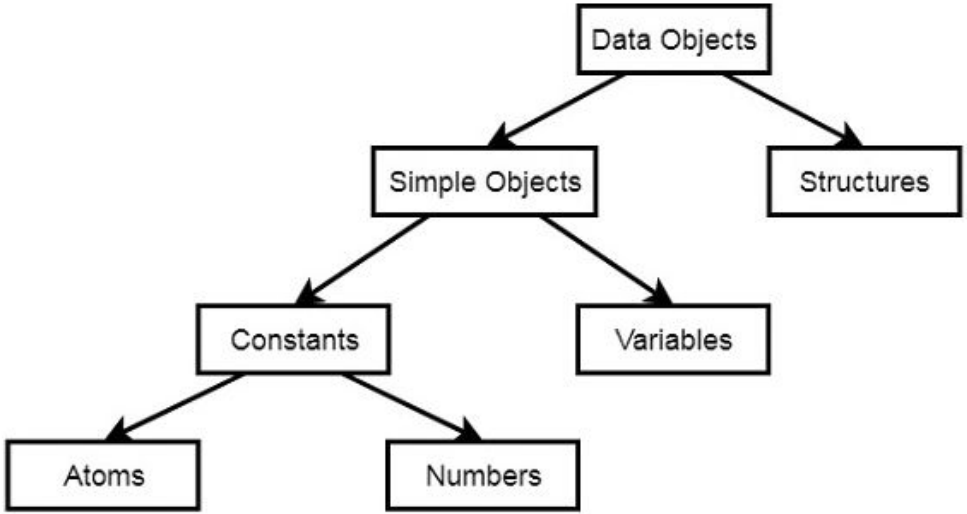
Example

- Lili is happy if she dances.
- Tom is hungry if he is searching for food.
- Andrew and Ilnar are friends if both of them love to play football.
- Ryan will go to play if school is closed, and he is free.

Solution:

```
happy(lili) :- dances(lili).
hungry(tom) :- search_for_food(tom).
friends(Andrew, Ilnar) :- lovesFootball(jack),
lovesFootball(bili).
goToPlay(ryan) :- isClosed(school), free(ryan).
```

Data Objects in Prolog



Data objects in Prolog

Arithmetic Expressions

The ***is built-in predicate*** is used in Prolog to force the **evaluation** of arithmetic expressions. If you just write something like X = 2 + 4, the result is to bind X to the unevaluated term 2 + 4, not to 6. Example:

```
?- X = 2 + 4.
   X = 2+4
```

If instead you write X *is* 2 + 4, Prolog arranges for the second argument, the arithmetic expression 2 + 4, to be evaluated (giving the result 6) before binding the result to X.

```
?- X is 2 + 4.
   X = 6
```

Comparison	Definition	Evaluates?
`X = Y`	succeeds if X and Y unify (match) in the Prolog sense	No
`X \= Y`	succeeds if X and Y do not unify; i.e. if `not (X = Y)`	No
`T1 == T2`	succeeds if terms T1 and T2 are identical; e.g. names of variables have to be the same	No
`T1 \== T2`	succeeds if terms T1 and T2 are not identical	No
`E1 =:= E2`	succeeds if values of expressions E1 and E2 are equal	Yes
`E1 =\= E2`	succeeds if values of expressions E1 and E2 are not equal	Yes
`E1 < E2`	succeeds if numeric value of expression E1 is < numeric value of E2	Yes
`E1 =< E2`	succeeds if numeric value of expression E1 is ≤ numeric value of E2	Yes
`E1 > E2`	succeeds if numeric value of expression E1 is > numeric value of E2	Yes
`E1 >= E2`	succeeds if numeric value of expression E1 is ≥ numeric value of E2	Yes
`T1 @< T2`	succeeds if T1 is alphabetically < T2	No
`T1 @=< T2`	succeeds if T1 is alphabetically ≤ T2	No
`T1 @> T2`	succeeds if T1 is alphabetically > T2	No
`T1 @>= T2`	succeeds if T1 is alphabetically ≥ T2	No

LISTS

- Lists:
  - []
  - [1, 2, 3]
  - [4, chan]
  - [pen, apple]
  - [[pen, pineapple], [apple, pen]]

- HEADS | TAILS
  - [1, 2, 3] % 1 - head, [2, 3] - tail
  - [H | T] % "read: H bar T" %
  - [H | T] matches with [1, 2, 3] as [1 | [2, 3]]

- [1, _, 3]
  - _% could be:
  - % [1, 2, 3]
  - % [1, x, 3]
  - % [1, [.....], 3]

Inbuilt predicates in Prolog

- **Identify if an element is part of a list:**
  - member(a, [a, b, c, d]).
- **Test for integers in a range:**
  - maplist(between(1, 3), [1, 2, 3]).

…

http://www.gprolog.org/manual/html_node/gprolog024.html

Exercise I

Write a program that compares two numbers.

Exercise I - Solution

Write a program that compares two numbers.

Solution 1.

```
find_max(X, Y, X) :- X >= Y.
find_max(X, Y, Y) :- X < Y.
```

Solution 2.

```
gte(X,Y) :- X > Y,write('X is greater').
gte(X,Y) :- X =:= Y,write('X and Y are same').
gte(X,Y) :- X < Y,write('X is smaller').
```

Exercise I - Extension

Extend the program to a program that recursively compares each element in the list to find the maximum value.

Exercise I - Extension Solution

Extend the program to a program that recursively compares each element in the list to find the maximum value.

```
max_list([X], X).
max_list([H|T], Max) :-
    max_list(T, RestMax),
    Max is max(H, RestMax).
```

Exercise II

Write a program that checks if two lists are equal.

Exercise III: Factorial Checker

- Write the program to check if the factorial of the first value is equal to the second value
- Expected outputs:
  - *?- factorial(3,6). // true*
  - *?- factorial(3,5). // false*

Exercise IV: Sum of elements

Define a program sum_list that takes a list of integers as its first argument and returns the sum of all the integers in the list as its second argument.

Exercise V: Palindrome

Define a program palindrome that takes a list as its argument and returns true if the list is a palindrome (i.e., reads the same forwards and backwards).


Q & A